

Szakdolgozat



Miskolci Egyetem

Mozgásrendszer kialakítása állapotgépek használatával

Készítette:

Halász Máté Sándor
Programtervező informatikus

Témavezető:

Dr. Hornyák Olivér

Miskolc, 2025

Miskolci Egyetem
Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék

Szám:

Szakdolgozat Feladat

Halász Máté Sándor (T1TNWL) programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: állapotgép, videójáték

A szakdolgozat címe: Karaktermozgatás állapotgépekkel a Godot játékmotorban

A feladat részletezése:

Tervezzen állapot gép alapú mozgásrendszert! Válasszon játékmotort a fejlesztéséhez! Legyen a játékos aktuális állapota nyílvántartva egy véges automata által és ezen automatán keresztül legyen szabályozva a lehetséges mozgások és interakciók! Legyenek a végrehajtható mozdulatok jól megfogalmazott előfeltételekhez kötve, amelyek az állapotgépen egyértelműen modellezhetők (például az ugrás végrehajtásának feltétele a "Grounded" (földön tartózkodó) állapot, míg a mozgás történhet földön és levegőben egyaránt)! Legyen szempont a könnyű bővíthetőség és átláthatóság, valamint az állapotok közötti zökkenőmentes átmenet (pl. "Grounded" → "Jumping", "Grounded" → "GroundedMoving", "Jumping" → "InAirMoving"). Mutassa be az előnyeit ennek a megközelítésnek! A fejlesztés menetét és eredményeit mutassa be!

Témavezető: Dr. Hornyák Olivér (egyetemi docens)

.....
szakfelelős

Eredetiségi Nyilatkozat

Alulírott **Halász Máté Sándor**; Neptun-kód: T1TNWL a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírással igazolom, hogy *Karaktermozgatás állapotgépekkel a Godot játékmotorban* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, év hó nap

.....
Hallgató

1. szükséges (módosítás külön lapon)
A szakdolgozat feladat módosítása nem szükséges

.....
dátum témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás): konzulens (dátum, aláírás):
.....
.....
.....

3. A szakdolgozat beadható:

.....
dátum témavezető(k)

4. A szakdolgozat szövegoldalt
..... program protokollt (listát, felhasználói leírást)
..... elektronikus adathordozót (részletezve)
.....
..... egyéb mellékletet (részletezve)
.....

tartalmaz.

.....
dátum témavezető(k)

5. bocsátható
A szakdolgozat bírálatra nem bocsátható

A bíráló neve:

.....
dátum szakfelelős

6. A szakdolgozat osztályzata

a témavezető javaslata:
a bíráló javaslata:
a szakdolgozat végleges eredménye:

Miskolc,

.....
a Záróvizsga Bizottság Elnöke

CD Használati útmutató

A CD mellékletben található 2 mappa: a Notes és a Codes.
A Notes mappán belül található meg a LaTeX forráskód. A Codes mappán belül pedig a Godot projekt minden hozzátartozó elemével együtt.

A Godot projekt beüzemelése

Ahhoz, hogy a Godot projektet be tudjuk üzemelni először is le kell tölteni a Godot Engine - .NET változatát. Ez a Godot hivatalos weboldalán mind elérhető [4]. Amikor elindítjuk a játékmotort a bal felső sarokban lesz 3 opció, ezek közül az "Import" opciót kell kiválasztanunk. Ezek után navigáljunk el a "Thesis/Codes/yagster" mappához, majd válasszuk azt ki és kattintsunk a "megnyitás" / "open" gombra. A Godot fel fogja ismerni, mint egy projekt és az importálás sikeres lesz. Ezek után kétszer az importált projektre kattintva a Godot meg fogja nyitni.

Tartalomjegyzék

1	Bevezetés	1
2	Véges állapotú automaták	3
2.0.1	Matematikai megközelítés	3
2.0.2	Programozói megközelítés	6
3	Koncepció	12
3.0.1	Játékmenet	12
3.0.2	Használt technológiák	13
3.0.3	Fejlesztési környezet választása	14
3.0.4	Játékmotor, programozási nyelv és fejlesztőkörnyezet	14
3.0.5	Vizuális világ megtervezése	15
3.0.6	Mozgásrendszer megtervezése	19
4	Megvalósítás	27
4.0.1	Az alapok	27
4.0.2	RigidBody vs CharacterBody	28
4.0.3	A játékos színtere	29
4.0.4	Változtatások az állapotgéphez	31
4.0.5	Az állapotok megvalósítása	32
4.0.6	Modellezés és Textúrázás	38

1. fejezet

Bevezetés

A videójáték készítés az elmúlt években egy dollármilliárdos iparággá nőtték ki magukat. Nem csak a rohamosan fejlődő technológiának, de a szoftveres ágon tett előre lépéseknek is köszönhetően. A 2000-es évek elején a videójáték gyártás mindössze a programozók (matematikusok) számára egy hobbi volt, ami az akkori hardverből a lehető legtöbbet való kihozásról szólt. Manapság a játékmotoroknak hála bárki elkezdhet játékokat gyártani, viszont ez nem azt jelenti, hogy sokkal egyszerűbb lenne a videójáték gyártók dolga. A rohamosan fejlődő ipar és a játékfejlesztés elérhetőségének a növekedése az elvárásokat is megnövelte a játékok iránt.

Mivel a számítástechnika és a matematika rokon tudományok, főleg a kezdetekben, amikor matematikus programozók voltak, ezért sok gyakorlatot alkalmazunk belőle. Egyike ezeknek a gyakorlatoknak az állapotgépek alkalmazása. Az állapotgépeket bővebben kifejtem majd a [2. fejezet](#)-ben. Megemlítésre méltóak, azon esetek ahol fontos a könnyű bővíthetőség és az, hogy ne kelljen újabb változókat létrehozni annak érdekében, hogy kiküszöböljünk egyes eseteket. A legnépszerűbb példa egy állapotgép használatára az a videójátékokhoz kapcsolódik. A kezdetekben ez nem volt olyan fontos, mivel maguk a játékok egyszerűbbek voltak, lásd: Doom, Wolfenstein, Elite (1984) ahol a legfontosabb a grafika megoldása és helyes kirajzolása volt. Manapság, ahol ún. "Omni-Movement" van a játékokban, ami, ahogyan Charles[9] is megfogalmazta a posztjában, egy olyan mozgásfajta, ahol a játékos teste a fejétől függetlenül mozog, tehát a test és a fej (kamera) mutathat két különböző irányba. Ezen esetben a játékosoknak közelről kell figyelni a mozgásukat és cselekedeteiket, és jól definiált struktúrákat kell létrehozni egyes mozgássorozatoknak. A játékosok képesek különböző mozdulatok végrehajtására, többek között: futás, ugrás, csúszás és vetődés. Ezek mind precíz inputokat követelnek meg a játékostól és az állapotok pontos

nyilvántartását. Ennek a kivitelezéséhez az állapotgépek használata elengedhetetlen, mind a mozgásrendszer kivitelezéséhez, mind az animációk helyes lejátszásához.

A témámat, viszont nem az Omni-Movement inspirálta, az én projektemhez csak példaképpen kapcsolódik, hogy demonstráljam egy sokkal bonyolultabb koncepción is a témámat és megmutassam, hogy az iparban is használatos. Sokkal inkább a Shibuya-punk esztétika[12] királya és a mai napig a Sega egyik legkevésbé elismert kiadása a: Jet Set Radio (2000), és az az által inspirált Team Reptile játék a: Bomb Rush Cyberfunk (2023). Ezekről sokan nem hallottak, viszont, ha azt mondom, hogy "Tony Hawk Pro Skater", akkor az már több embernek fog ismerősen hangzani. A koncepció ugyan az mind a kettőnél: gurulj és csinálj menő trükköket (miközben próbálsz magad nem összetörni, természetesen). Ha a felszínt nézzük is már eléggé bonyolultnak néz ki a helyzet, mind mozgás, mind pontszám számítás szempontjából; és elkezd járni az agyunk azon, hogy mégis mennyi állapotot kellett nyomon követniük a fejlesztőknek, hogy ezeket megvalósítsák. Annak érdekében, hogy egy kicsit mélyebbre ássak és megválaszoljam ezt a kérdést úgy döntöttem, hogy magam is nekiállok és létrehozok egy mozgásrendszert, amiben tesztelhetjük, hogy mennyire is érné meg állapotgépeket használni, miért és mennyivel eredményezne szebb, jobban strukturált, átláthatóbb, könnyebben bővíthető kódot, és gyorsabb programot, mint egy hagyományos if-and megoldás.

A szakdolgozat végére szeretnék egy teljesen működőképes és játszható mozgásrendszer-prototípust elkészíteni, amely bemutatja az állapotgépek fontosságát a mozgásrendszerek kialakításában és a játékfejlesztés más terein. Mind ezt a Shibuya-punk esztétikában többnyire magam által elkészített modellekből.

2. fejezet

Véges állapotú automaták

Ahogy az informatika fejlődött és egyre több matematikai koncepciót adaptált, hogy segítse a fejlesztést, úgy távolodott is tőle. Meglepő módon nem kell ismernie az embernek a pontos matematikai definícióit egyes koncepcióknak annak érdekében, hogy használja őket fejlesztés során. Ez leginkább itt fog majd érződni, az automaták terén.

2.0.1 Matematikai megközelítés

A véges állapotú automata (Finite State Machine, avagy FSM) egy matematikai model, amely képes ábrázolni a dinamikus viselkedését egy rendszernek véges számú állapotot alkalmazva, valamint ezen állapotok közötti átmenéseket, és az átmenetekhez szükséges cselekvéseket. Bármely adott pillanatban egy rendszer egy bizonyos állapotot vesz fel és egy átmenet egy válasz egy bizonyos cselekvésre, vagy történésre.

A véges állapotú automaták kategorizálhatóak **determinisztikus** és **non-determinisztikus** automatákra a viselkedésük szempontjából:

- **A determinisztikus véges automaták (DFSM)** esetében minden állapotnak van egy egyedi átmenete minden lehetséges bemenetre (input). A következő állapotot csakis a jelenlegi állapot és a kapott input dönti el, amely egy kiszámítható és egyértelmű viselkedést eredményez.
- **A non-determinisztikus automaták (NDFSM)** esetében több átmenet létezhet egy inputra és a jelenlegi állapotra. A rendszer következő állapota nem egyedien lesz eldöntve, ami rugalmasságot eredményez, de egyben kiszámíthatatlanságot és komplexitást is bevezet.

A mi esetünkben csak determinisztikus automatákkal fogunk foglalkozni, de érdemes volt megemlíteni a non-determinisztikus automatákat, mivel mate-

matikai és tervezésbeli jelentőséggel bírnak.

Ahhoz, hogy tovább tudjunk beszélni róla, először definiáljuk. Legyen **M** egy determinisztikus véges automata, ekkor igaz, hogy:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Ahol:

- 1 Q , egy véges halmaz, az állapotok halmaza
- 2 Σ , egy véges halmaz, az ABC
- 3 $\delta, Q \times \Sigma \rightarrow Q$ az állapotátmenet függvény
- 4 $q_0 \in Q$ a kezdeti állapot
- 5 $F \subset Q$ a végállapotok halmaza

Egy determinisztikus automata felismer (elfogad) egy jelsorozatot, ha $w = x_1, x_2 \dots x_n$ Σ -beli jelek sorozata. M felismeri w -t, ha létezik olyan $r_0, r_1, \dots, r_n$ Q -beli állapotok sorozata, hogy:

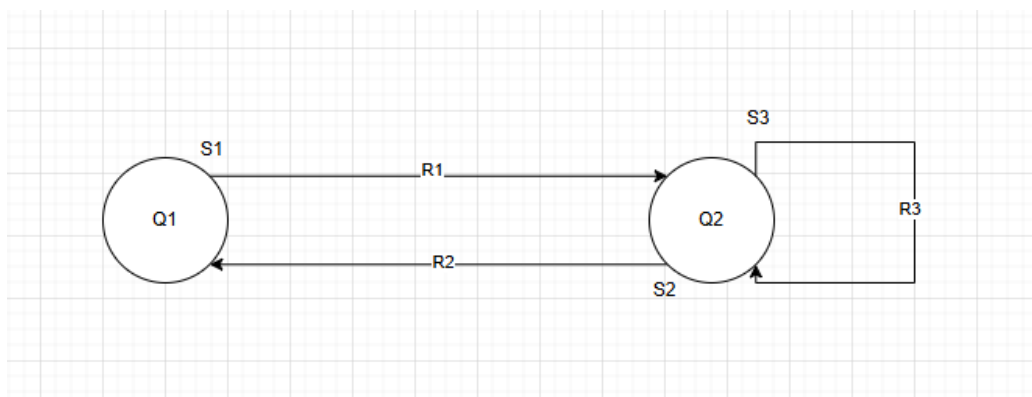
- $r_0 = q_0$
- $\delta(r_i, x_{i+1}) = r_{i+1}$, ahol $i = 1, \dots, n - 1$
- $r_n \in F$

Definíció: M DFMSM felismeri az A nyelvet, ha

$$A = \{w \mid M \text{ felismeri } w - t\}$$

Jelölés: $A = L(M)$, avagy "A, az M automata nyelve"

Nézzük meg ezeket a részeket egy kicsit részletesebben is.



2.1. ábra: Példa állapotgép

Egy egyszerű példa egy ajtó nyitására, nyitva tartására és csukására, amit Georges Ltlief papírjából[10] szereztem.

Állapotok:

- Az állapotok segítségével tudjuk megmondani, hogy milyen feltételek és módok mellett operál a rendszerünk a jelenlegi pillanatban. Minden állapot egy egyedi viselkedést ír le, amit a rendszer produkál. Fontos, hogy az állapotok száma *egy véges halmaz.* - $Q_1, \dots, Q_n$ -nel jelöljük az állapotokat, amikor vizualizáljuk az automatánk

Átmenetek:

- Az átmenetek írják le a rendszer válaszait egyes bemenetekre, vagy eseményekre, amelyek átlölik az automatát az egyik állapotból a másikba.
- Csak bizonyos ingerek (stimuli, a példában S_1, S_2, S_3) eredményezhetnek átmenetet a rendszeren belül, olyanok amelyek benne vannak az automata **nyelvében**.
- Az átmeneteket mindig irányított nyilakkal jelöljük az ábrákon.

Ingerek:

- Az ingerek olyan események, vagy bemenetek, amelyek átmenetet eredményeznek az automatán belül. Lehetnek pl. felhasználói parancsok, szenzor érzékelések, hálózati üzenetek, stb.

Válaszok:

- Egy automata válasza (vagy kimenete) mindig egyedi az állapotot nézve. Ezek lehetnek: egy üzenet kiírása/elküldése, egy egész algoritmus lefuttatása, vagy csak egy változó felülírása.
- A példában a válaszokat (responses) az R_1, R_2, R_3 ábrázolja, az S_1, S_2, S_3 ingerekre.

2.0.2 Programozói megközelítés

Programozás szempontjából az automaták egy nagyon hasznos része az informatikának, mivel végtelenül egyszerűvé teszi egyes rendszerek nyomonkövetését, megtervezését és automatizálását. Egy rendszeren belül, vegyünk most egy játékot példának, akármilyen objektumnak lehet állapota és, ahogyan ezt már említettem, az állapottól függően fognak változni az objektumok viselkedése is, például:

- A pálya időjárása (eshet, süthet a nap, fújhat a szél),
- a karakter fizikai állapota (futhat, ugorhat, csúszhat, támadhat),
- a fegyver állapota, amivel támadni akarunk (készletben, töltődik újra, elhasználva).

Vegyük észre, hogy az állapotok kihathatnak egymásra is, példaképpen: A karakterrel éppen csúszás közben vagyunk és a csúszda végén ott van egy ellenfél, akit meg akarunk támadni, ezért elővesszük a fegyverünket, de ki van fogyva és ez átrak minket az "újrátöltés" animációba, ami miatt csak belecsapódunk az ellenfélbe, a támadás helyett. Azzal, hogy mindig tisztában vagyunk vele, hogy milyen állapotban vannak a világunk egyes részei, megkönnyebbítjük a magunk dolgát a hibakeresésben és a játékosok dolgát azzal, hogy nem zavarjuk őket össze.

Amikor egy rendszernél állapotgépeket használunk, akkor általában kéz-a-kézben jár a "kompozíció", és az "öröklődés" használata mint Objektum Orientált Programozásban használt tervezési minta. Ahhoz, hogy jobban megértsük hogyan is nézne ez ki kódban kezdés képpen készítettem egy mintaprogramot, amely mintaprogram egy egyszerű jelzőlámpa működését mutatja be.

Elsőként is létrehoztam egy vázat az állapotok számára, minden állapot egy külön Node lesz a Godot-n belül. Ez egy teljesen absztrakt osztály, amit valójában nem fogunk futtatni mint script, ezt csak örökölni fogják az egyes állapotaink.

```

1 using Godot;
2 using Godot.Collections;
3
4 public partial class State : Node
5 {
6     public StateMachine fsm;
7
8     public virtual void Enter() {}
9     public virtual void Exit() {}
10
11     public virtual void Update(double delta) {}
12     public virtual void PhysicsUpdate(double delta) {}
13
14     public virtual void HandleInput(InputEvent @event) {}
15
16 }

```

Lehet, hogy megfordul a fejünkben a gondolat: "De, akkor miért nem egy absztrakt osztály?" Azért mert, akkor nem tudnánk felvenni az állapotokat egy Dictionary-be, erre később vissza is térek. Egy állapotnak 5 metódusa van: Enter, Exit, Update, PhysicsUpdate, HandleInput. Ezek mind a megfelelő pillanatban lesznek meghívva, lehet, hogy nem mindegyik állapot enged majd inputot feldolgozni, vagy lehet, hogy nem mindegyikre fog kihatni a fizika, viszont ez a metódus-ötös az alapja a legtöbb állapotnak. Az "Enter" metódusban az állapot ellenőrizni fog egy pár feltételt és fel fogja építeni az állapotban végrehajtható cselekvésekhez szükséges környezetet. Az "Exit" egyértelműen ennek az ellenkezőjét fogja csinálni. Az "Update" és "PhysicsUpdate" metódusok mind az eltelt (delta) idő szerint fognak végrehajtani eseményeket, pl.: a levegőben töltött idő mérése, esési sebesség, féktáv, stb. Végül a HandleInput akármilyen bemenetre fog reagálni és azokat lekezelni. Ha például a karakter éppen a levegőben van és elkezd mozogni, akkor azt elkapja a HandleInput metódus és megoldja, hogy ne csak egyhelyben tudjon a karakter ugrani, de különböző irányokba is. A metódusokon kívül van még egy változónk, ami az "fsm".

```

1 using Godot;
2 using System.Collections.Generic;
3
4 public partial class StateMachine : Node
5 {
6     [Export] public NodePath initialState;
7
8     private System.Collections.Generic.Dictionary<string,
9         State> _states;
10     private State _currentState;

```

```

10
11 public override void _Ready()
12 {
13     _states = new Dictionary<string, State>();
14     foreach(Node node in GetChildren())
15     {
16         if(node is State s)
17         {
18             _states[node.Name] = s;
19             s.fsm = this;
20             s._Ready();
21             s.Exit();
22         }
23     }
24     if(initialState != null)
25     {
26         _currentState = GetNode<State>(initialState);
27         _currentState.Enter();
28     }
29     else
30         GD.Print("Initial State not set");
31 }
32
33 public override void _Process(double delta)
34 {
35     _currentState.Update((float)delta);
36 }
37
38 public override void _PhysicsProcess(double delta)
39 {
40     _currentState.PhysicsUpdate(delta);
41 }
42
43 public override void _UnhandledInput(InputEvent @event)
44 {
45     _currentState.HandleInput(@event);
46 }
47
48 public void TransitionTo(string key)
49 {
50     if(!_states.ContainsKey(key) || _currentState ==
51         _states[key])
52     {
53         return;
54     }
55     GD.Print("Transitioning from ", _currentState.Name.
56         ToString(), " To ", _states[key].Name.ToString());

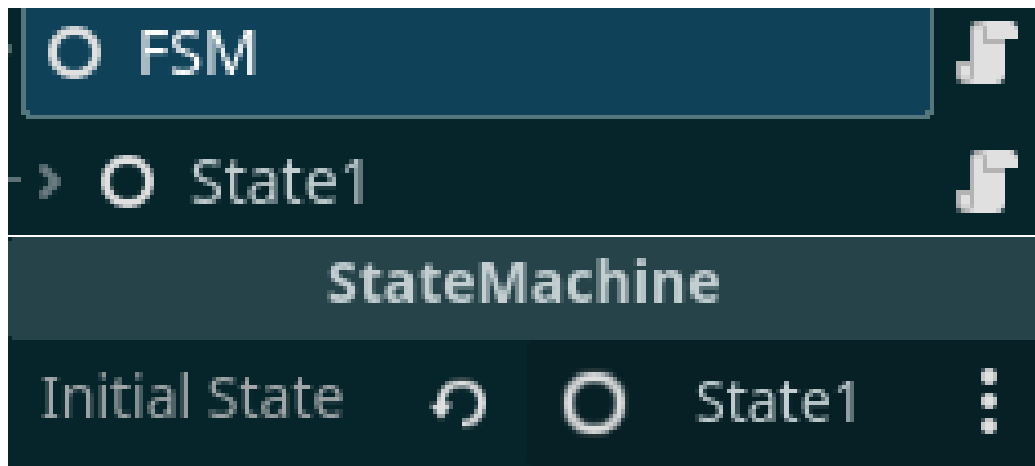
```

```

57     _currentState.Exit();
58     _currentState = _states[key];
59     _currentState.Enter();
60 }
61 }

```

Ez itt a "StateMachine" osztályunk. A State.cs-el ellentétben ez a script hozzá lesz csatolva egy Node-hoz és futtatva lesz, ő lesz az állapotgépünk gyökere, mint egy vezérlő. Minden más Node, ami állapot, hozzá fog tartozni. Itt sok minden történik, de leegyszerűsítve az "initialState" változónk, egy "NodePath", avagy Node-hoz vezető út. Az [Export], ami elé van írva egyszerűen annyit jelent, hogy az editor-ból is meg tudjuk változtatni annak értékét.

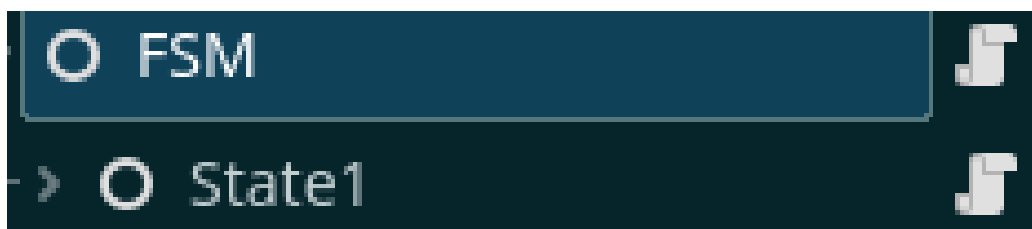


2.2. ábra: Az alap állapot beállítása

Az állapotgépünknek minden képpen kell egy kiindulópont, egy első állapot és ezt mi választjuk ki itt. Továbbá egy Dictionary-ként tárolom a helyes állapotokat és lementem egy "_currentState" változóba a jelenlegi állapotot, hogy mindig számon tudjam tartani. A _Ready() metódusban, ahogyan látjuk végigiterálunk minden "Node" típusú változón. Itt térek vissza az előbbi kijelentésemre, muszáj volt megtartani a "State" osztályt Node-ként, mivel a "GetChildren()" beépített metódus Node típusú változókat ad vissza. A _Process(), _PhysicsProcess() és _UnhandledInput() metódusok mind meghívják a jelenlegi állapot saját metódusát ezekre az alkalmakra. Végül a TransitionTo(string key) metódus felelős az állapotok közötti váltásért. A "kulcs" ebben az esetben nem más mint egy string, ami magának a Node-nak is a

neve. Ez a jobb olvashatóság és struktúra érdekében volt így megoldva.

Példaképpen:



2.3. ábra: A befejezett állapotok fája

Az állapotgépünk áll négy állapotból. Mindegyik állapothoz hozzá van rendelve egy fényforrás és egy időzítő.

```
1 public partial class State3 : State
2 {
3     public override void Enter()
4     {
5         GetNode<Node3D>("Light").Visible = true;
6         GetNode<Timer>("Timer").Start();
7     }
8
9     public override void HandleInput(InputEvent @event)
10    {
11        if(@event is InputEventKey eventKey)
12        {
13            if(eventKey.Pressed)
14            {
15                if(eventKey.Keycode == Key.Key1)
16                {
17                    EmitSignal(SignalName.Transition, "State1");
18                }
19                if(eventKey.Keycode == Key.Key2)
20                {
21                    EmitSignal(SignalName.Transition, "State2");
22                }
23                if(eventKey.Keycode == Key.Key3)
24                {
25                    EmitSignal(SignalName.Transition, "State3");
26                }
27                if(eventKey.Keycode == Key.Ctrl)
28                {
29                    EmitSignal(SignalName.Transition, "State4");
```



```
30         }
31     }
32 }
33 }
34
35 public override void Exit()
36 {
37     GetNode<Node3D>("Light").Visible = false;
38     GetNode<Timer>("Timer").Stop();
39 }
40
41 private void _OnTimerTimeout()
42 {
43     EmitSignal(SignalName.Transition, "State1");
44 }
45 }
```

A "State3"-nak elnevezett állapotban vagyunk jelenleg, ami ebben az esetben a "Piros" állapot a jelzőlámpánkon. Amikor erre az állapotra váltunk, akkor a State3-hoz csatolt fényforrást láthatóvá kell tennünk, valamint el kell indítanunk az időzítónket, hogy nehogy benne ragadjunk. Amikor az időzítő lejár, a gép átkerül a "State1"-be, ami a mi estünkben a "Zöld". Amikor átlépünk a következő állapotba fontos, hogy az előző állapotból sikeresen kilépjünk és megszüntessünk minden olyan dolgot, ami bezavarna más állapotok működésébe. Ezért az állapotgépünk meg fogja hívni a State3-nak az "Exit()" metódusát, amelyben is elrejtjük a fényforrását és megállítjuk az időzítőjét, arra az esetre, ha nem állna meg magától.

Ezen felül van egy egyszerű switch statement, ami felhasználói bemenetre reagál ezzel lehetővé téve az állapotok közötti egyszerű ugrálást.

3. fejezet

Koncepció

Az évek alatt sok olyan játékkal játszottam, amelyek mélyen kidolgozott mozgásrendszerekre vannak építve, ilyen műfajok például a Movement-Shooter, bizonyos Sport játékok többnyire azok, amik gördeszkákra, görkorcsolyákra és a hasonlókra épülnek. Fontosnak tartottam itt megemlíteni a Movement-Shooter műfajt, mivel az ebbe tartozó játékok szerettették meg igazán velem a szofisztikált mozgásrendszereket és belőlük tanultam sokat. Ezen játékok tökéletesen összehangolják a finom mozdulatokat az aréna-lövöldék (mint például a Quake) gyors döntéshozatalával és precíz célzás igényével. Ezeket a műfajok csupán motivációként hoztam fel, ha esetleg a kedves olvasónak megjönne a kedve jobban belemenni a gyorsabb videójátékok világába.

3.0.1 Játékmenet

A játékmenet nem komplikált, mivel ez túlmutatna a projektem lényegén. A játékos képes mozogni minden irányba (jobb, bal, előre, hátra), képes ugrani, valamint a levegőben mozogni. Ez kombinálva a játékos képességével, hogy "meglovagoljon" csöveket egy elég egyszerű, de erős alapot ad egy tényleges nagyobb játék elkészítéséhez. Mindezek ellenére, úgy érzem, hogy le kell szögezmem, hogy a színtér maga csupán arra szolgálna, hogy be tudjam mutatni a mozgásrendszert és vizualizálni tudjam az állapotgépet munka közben. Az állapotgépet úgy terveztem meg, hogy akármilyen projektben, amely a Godot játékmotoron belül készült, használható legyen. A saját felhasználásom egy olyan mozgásrendszert alakított ki, ahol valamilyen gurulásra vagy csúszásra alkalmas eszköz van (pl.: gördeszka, görkorcsolya, síléc, jégkorcsolya) a játékos karakteren. Ehhez a Tony Hawk's Proskater játéksorozatot és Bomb Rush Cyberfunk / Jet Set Radio játékmenetét és mozgásrendszerét vettem alapul. Azzal az eltéréssel, hogy a saját projektben megnyerési pont nincs, mivel ez túlmutatott a projektnek megszabott

feladaton.

3.0.2 Használt technológiák

A használt technológiákat két részre osztanám fel: **szoftver** és **hardver**. Most egyenlőre, csak egy rövidebb felsorolást tennék a technológiákról és a későbbiekben, amikor a megfelelő részekhez érek, bővebben beszámolok róluk.

Szoftver szempontjából:

- [Blender](#)[2]-t használtam a 3Ds modellek elkészítéséhez.
- [Krita](#)[5]-t használtam a Modellek textúráinak megrajzolásához és [SLK_img2pixel](#)[7]-t használtam a színek állításához.
- A hangok megvágásához [Audacity](#)[1]-t és [LMMS](#)[6]-t használtam.
- A diagramok megszerkesztéséhez [Draw.io](#)[3]-t használtam.
- A verziókövetéshez Git-et használtam.

Hardver szempontjából a szoftvert két különböző rendszeren teszteltem és fejlesztettem:

- Asztali Számítógép:
 - Windows 10
 - Processzor: AMD Ryzen 5 7600X 6-Core
 - AMD Radeon RX 6600
 - Memória (RAM): 32GB
- LENOVO IdeaPad Slim 3 15AMN8:
 - Kubuntu 25.10
 - Processzor: 8 x AMD Ryzen 5 7520U with Radeon Graphics
 - Videókártya: AMD Radeon 610M
 - Memória (RAM): 16GB

Megjegyezném, hogy ez nem jelenti azt, hogy csak ezen gépigényeket elérő rendszereken működne a rendszer, egyszerűen csak ezeken tudtam kipróbálni. Valamint a digitális koncepció rajzok elkészítéséhez és a textúrák megalkotásához az: XP-Pen Deco 02-öt használtam.

3.0.3 Fejlesztési környezet választása

A fejlesztési környezet választásakor több szempontot is figyelembe kellett vennem:

- Elérhetőség: Mivel a fejlesztést két különböző rendszeren, két különböző operációs rendszer alatt végeztem fontos volt számomra, hogy a fejlesztési környezet elérhető legyen mind a két operációs rendszeren.
- Programozási Nyelv Támogatása: A Godot-n belül van számos programozási nyelvhez támogatás, ezek a:
 - GDScript a Godot saját script nyelve,
 - C# a Godot Mono támogatásával,
 - Valamint C és C++ számos kiegészítő támogatásával.

Fontos volt, hogy ezek közül az egyikhez legalább támogatást nyújtson a fejlesztési környezet.

- Kényelem: A fejlesztési környezet által nyújtott kiegészítési, valamint emlékeztető segédletek fontosak voltak, mivel a rendszer sok hasonló kódrészt tartalmaz (pl. Dictionaries, Leképezések, stb.).
- **Integráció a motorral**: Ez egy bónusz pont a listán, de sokat segít, ha a fejlesztési környezet integrálja a játékmotort az egyszerű dokumentáció olvasás és metódus / függvény hívás érdekében.

3.0.4 Játékmotor, programozási nyelv és fejlesztőkörnyezet

A játékmotor, programozási nyelv és fejlesztőkörnyezet megválasztása során fontos volt számomra a platformfüggetlenség, kompatibilitás és a hordozhatóság.

- Kompatibilitás / Platformfüggetlenség: Mivel a szoftvert két különböző operációs rendszeren fejlesztettem ezért elengedhetetlen volt az, hogy az összes projektfejlesztéshez használt program elérhető legyen mind egy Linux környezet, mind egy Windows környezet alatt. A játékmotorok, amik mindig szóba jönnek az a Unity és az Unreal Engine, viszont ezeknél az motoroknál a licenz kérdése is szóba jön. Nem egy utolsó szempont az se, hogy ezzel a rendszerrel én majd a jövőben tovább dolgozhassak akármilyen fennakadás vagy hátráltatás nélkül.

- Tapasztalat: Fontos volt a meglévő tapasztalat a választott technológiával, hogy a lehető legjobbat ki tudjam hozni a projektből reális időn belül. Egy új rendszer vagy programozási nyelv megtanulása időt vett volna el, amit fejlesztésre tudtam volna használni.

Így mindezeket a szempontokat összevetve a választásaim végül a: Godot motorra, C# programozási nyelvre és a Visual Studio Code fejlesztőkörnyezetre esett.

3.0.5 Vizuális világ megtervezése

A vizuális világ, amint már említettem a Shibuya punk esztétikára alapszik. [3.1] Agresszív vonalak, elmosott, neon színek erőteljes használata és dinamikus kompozíciók definiálják. Műfajalkotó játéknak számít a Jet Set Radio (amire mostantól **JSR**-ként fogok hivatkozni).



3.1. ábra: Demonstráció az agresszív vonalakra

A JSR 2000-ben jött ki a DreamCast konzolra. És korszakalkotó volt a játék vizuális világa, [3.2] játékmenete és mozgásrendszere. Százezreket fogott meg, sajnos évekre rá sem jött ki olyan játék, ami el tudta volna csípni, hogy mi is tette a játékot annyira ikonikussá. A játékmenetében is megtartja a dinamikusságot és az agresszív vonalakat, ami a játékosban azt az érzést kelti, hogy gyorsabb és pontosabb mint valójában.



3.2. ábra: Képpillanat a JSR játékmenetéről

Ahhoz, hogy pontosan replikálni tudjam ezeket a tulajdonságokat mélyen bele kellett magam ásnom a játék világába, de mivel egy eléggé régi játékról van szó, nem könnyen hozzáférhető. Szerencsére erre a problémára megoldást találtam egy 2023-mas ún. "Szerelmi Vallomásban" (Love Letter), a [Bomb Rush Cyberfunk](#)-ban [3.3] (amire mostantól **BRCF**-ként fogok hivatkozni).



3.3. ábra: Reklámkép a Bomb Rush Cyberfunk-ról

A játék teljes mértékben a JSR-t veszi alapul és megpróbálja replikálni a játékot annyira, amennyire csak tudja [3.4], miközben újít és iterál a meglévő koncepciókon. A játékmenet kifinomultabb, mint a JSR-nak, de képes megtartani azt az érzést, amit annak idején az elődje keltet. A Team Reptile leírása alapján: "Dion Koster elmevilágában, ahol saját-stílusú bandák személyi jetpack-ekkel vannak felruházva, a graffiti művészet új magaslatoiba szárnyal. Vágj bele te is a világba és táncolj, fess, trükköz, verd vissza a korrupt rendőrséget és foglald el a város minden zegét-zugát egy alternatív jövőben, amit életre kelt Hideki Naganuma zenéje" [13].



3.4. ábra: Játékbeli kép a Bomb Rush Cyberfunk-ról

Látszatra a két játék ugyan azt a stílust tartja, azzal a különbséggel, hogy míg a JSR egy földhöz-ragadtabb játékmenetet és világot ad át, mint ha egy, a 20-as éveiben lévő lázadó punk tini lennél, addig a BRCF egy sokkal sci-fi-sebb világba dob minket, ahol mindenkinek lehet egy jetpack a hátán, ami eszméletlen sebességekre gyorsít fel minket és rásegít a trükkjeinkre. A vizuális világ megtervezésében tehát ezeket a szempontokat vettem alapul és ezek szerint alakítottam a világot és a karaktert.

3.0.6 Mozgásrendszer megtervezése

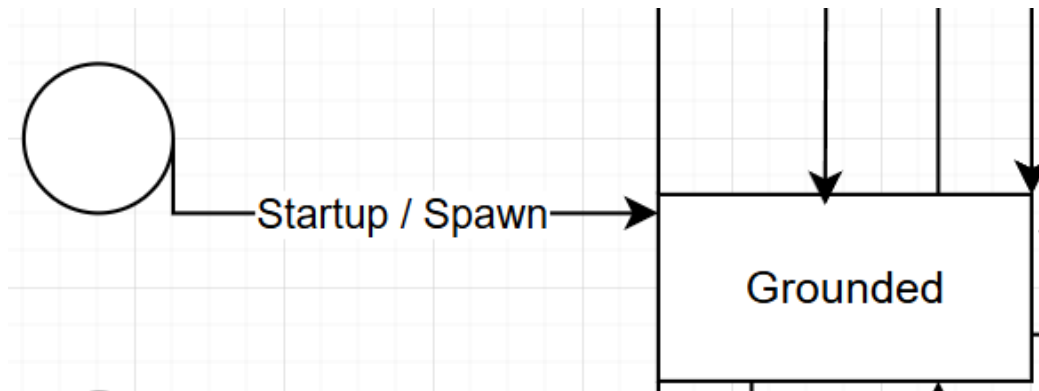
Most, hogy a vizuális világ megbeszélésre került, ideje áttérni arra, ami igazán fontos: a mozgásrendszer.

Ahhoz, hogy egy minél folyékonyabb mozgásrendszert tudjak alkotni sok tervezés és előre gondolás kellett. A véges automaták egyik előnye az, hogy nagyon könnyen bővíthetőek, így ha el is rontottam volna valamit, vagy esetleg kellett volna még egy állapot ahhoz, hogy az animáció vagy a mozgás jól jöjjön ki, akkor azt könnyen megtehettem volna. Ennek a tulajdonságnak köszönhetően volt egy kis mozgásterem a kísérletezésre, ami újoncként nagy előny.

Állapotok

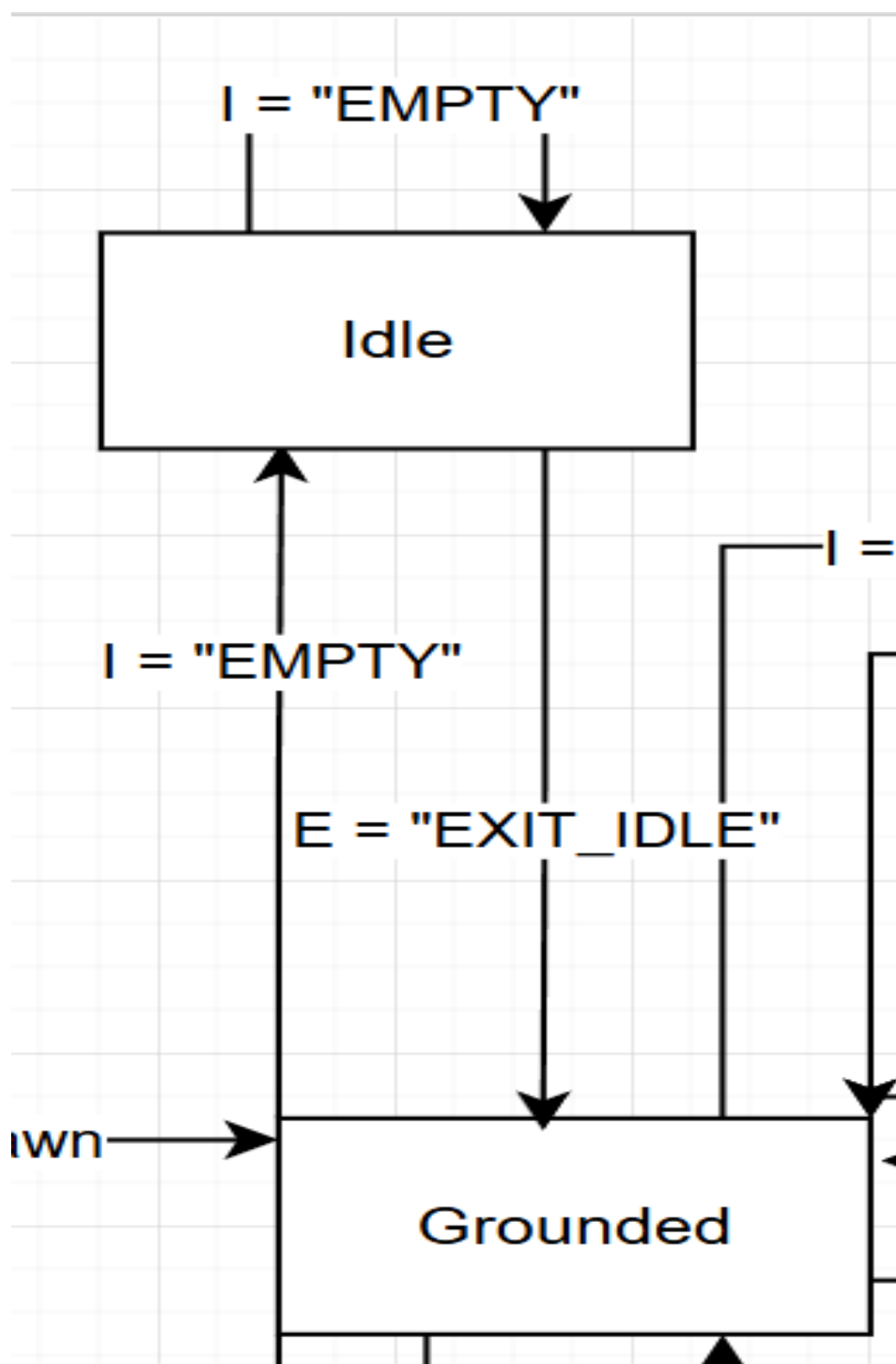
Át kellett gondolnom, hogy hogyan is szeretném, hogy kinézzen egyes mozdulat és, hogy egyes, a játékos által megadott bemenetre, milyen mozdulat lehet végrehajtható, bizonyos időkben. Egyszóval, a mozgás-láncokat meg kellett alkotnom. Viszont, nem csak a játékos kezdeményezhet állapotváltozásokat, hanem a program maga is. Előfordulhatnak események, amelyek befolyásolják a játékos mozgását és egyben az állapotát is. Ahhoz, hogy megkülönböztessem a program eseményeit a játékos bemeneteitől elneveztem azokat az eseményeket, amik a játékos irányítása fölött vannak "E"-nek, mint "Event" és a játékos bemeneteit "I"-nek, mint "Input". Most már tárgyalásra kerülhet az állapot diagram.

Minden a belépéssel vagy leidezéssel (spawn) kezdődik. A felhasználó itt kapja meg az irányítást a karaktere fölött, ezt a [3.5] kép demonstrálja.



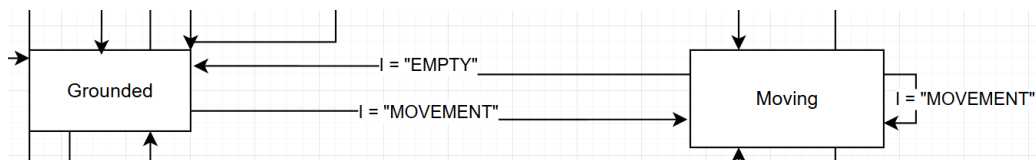
3.5. ábra: A program elindítása / a karakter újraéledése utáni állapot

A karakter minding egy szilárd, állható talaj fölött éled le, vagy éled újra, ezzel garantálva az állapotok közötti folytonosságot. Az alap állapot minden esetben az ún. "Idle", avagy nyugalmi állapot lesz, ahol a karakter egy állható, szilárd talajon van és játékos nem ad meg bemenetet, ennek a vizualizálása a [3.6] képen látható.



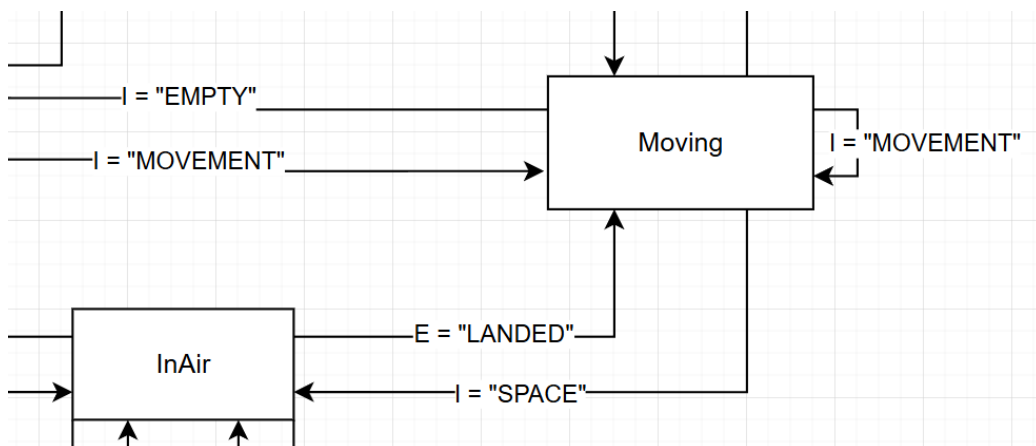
3.6. ábra: A karakter²¹ egyhelyben állása

Ebben az állapotban egy nyugalmi animáció játszódik le, ami életszerűbbé teszi a karaktert. A szilárd talajon való állásból (Grounded) elindulhat a játékos valamely mozgás gomb lenyomásával az egyik irányba ezzel átlökve az automatát a mozgásban levés állapotába (Moving), ahogyan azt a [3.7] kép is mutatja.



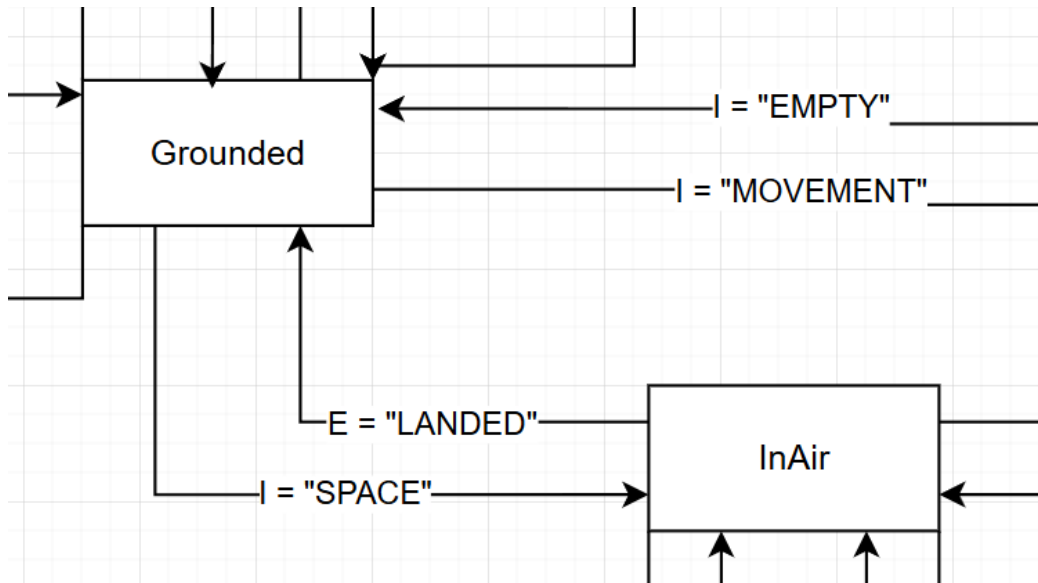
3.7. ábra: A karakter állható talajon való mozgása

Ebből az állapotból a játékos visszatérhet a Grounded állapotba ha abba hagyja a mozgást. Viszont, ha a játékos mozgás közben lenyomja az ugrás gombot (ami ebben az esetben a "SPACE"), akkor átlöki az automatát a levegőben lévő állapotba (InAir), ami az alábbi [3.8] képen látható.



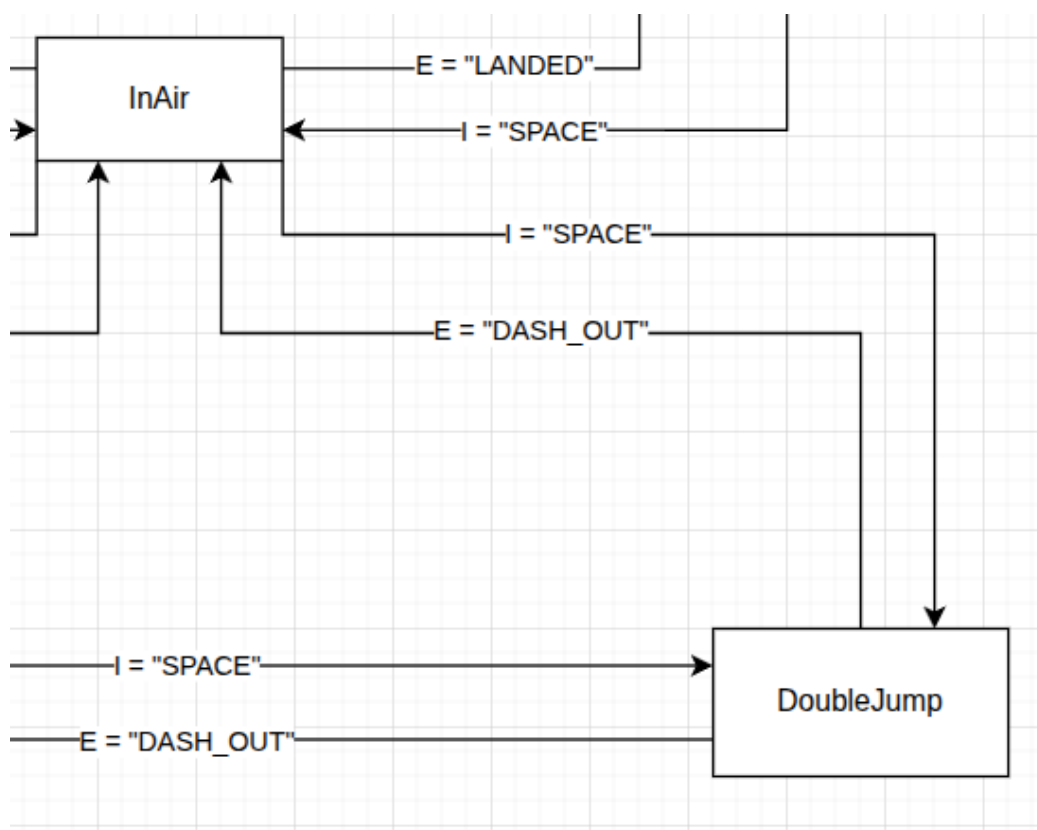
3.8. ábra: A karakter mozgásból való ugrása

Az ugrás időhöz van kötve, amit a gravitációs vonzóerőből és a sebességből számol ki a program. Amint ez az idő lejár és a karakter földet ér a program kiad egy "földet ért" (LANDED) eseményt, amiből, abban az esetben ha a játékos még mindig mozog, akkor visszatér a Moving állapotba. Amennyiben, viszont a játékos már nem ad meg bemenetet a mozgásra, akkor az InAir állapotból visszatérünk a Grounded állapotra, amit a [3.9] kép demonstrál.



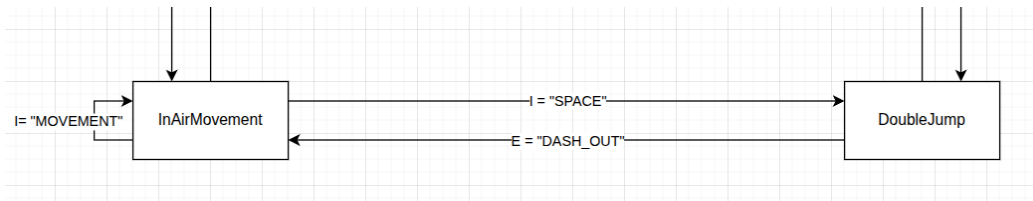
3.9. ábra: A karakter állható talajról való ugrása

Vegyük észre, hogy a Grounded állapotból is el lehet jutni az InAir állapotba, abban az esetben, ha a játékos nem ad meg mozgásra bemenetet, viszont ugrásra igen. Ha a játékosnak viszont kell még egy kis idő a levegőben, akkor még egyszer megnyomva az ugrás gombot, akkor átlöki az automatát a dupla ugrás (DoubleJump) állapotba, ahogy azt a [3.10] képen is lehet látni. Ebben az állapotban megtartja a momentumát a játékos csak a felfele mutató vektorhoz adódik hozzá megint az érték.



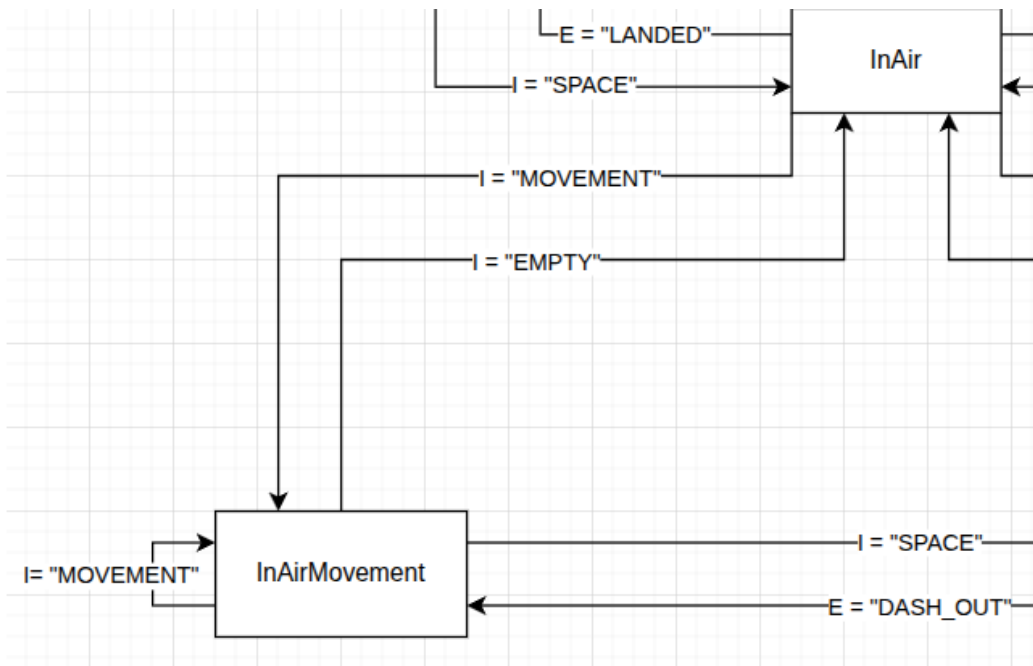
3.10. ábra: A karakter levegőből való ugrása

Vegyük észre, hogy a játékosnak most már nincs irányítása a karakter levegőben maradása fölött, mivel kaptunk egy eseményt, ami megmondta nekünk, hogy nincs több ugrásunk ("DASH_OUT"). Abban az esetben ha a játékos a dupla ugrás után tovább szeretné mozgatni a karaktert a levegőben valamilyen okból kifolyólag, akkor a mozgás billentyűket lenyomva át tudja lökni az automatát a levegőben lévő mozgás (InAirMovement) állapotba, ahogyan azt a [3.11] kép is mutatja. Amíg a játékos lenyomva tartja a mozgás gombokat, addig ebben az állapotban marad.



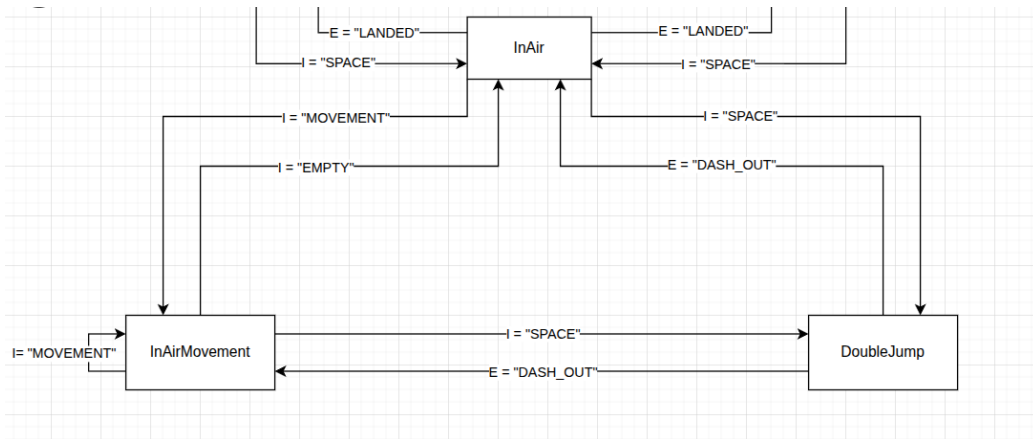
3.11. ábra: A karakter levegőben való mozgása egy dupla ugrás után

Amennyiben a játékos abba hagyja a mozgást, és még nem futott ki a levegőben tölthető időből az automata vissza lesz lökve a levegőbeli állapotba (**InAir**). Viszont, ha ismét el kezd mozogni, akkor vissza lesz lökve a levegőben lévő mozgás állapotába, ezzel befejezve a kört, ahogy az az alábbi [3.12] ábrán is látható.



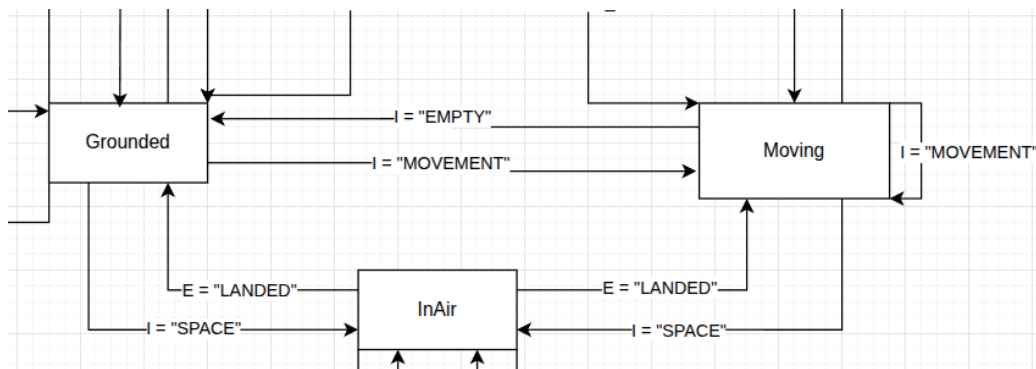
3.12. ábra: A karakter levegőben való mozgása és esésbe való visszatérése

Érdemes megjegyezni, hogy az eddig felsorolt állapotok össze vannak kötve egymással. Ezeket mozgás hármaskoknak hívom. A legtöbb mozgásfajtát le lehet bontani kisebb csoportokra amiknek van hozzáférése 1 vagy 2 olyan állapothoz, amely átviszi őket egy másik csoportba. Példa képpen a levegőben történő mozgáscsoport hármasa a [3.13] képen látható.



3.13. ábra: A levegőben történő mozgáscsoport

Valamint a földön történő mozgáscsoport hármasa a [3.14] képen.



3.14. ábra: Az állható talajon történő mozgáscsoport

Ezen mozgáscsoportok közötti átjárást az **ugrás** cselekvés biztosítja, ami további mozgáslehetőségeket tár fel. Név szerint a dupla ugrás (Double-Jump) és a levegőbeli mozgás (InAirMovement), amikbe a már megszokott "MOVEMENT" cselekvés és "SPACE" gomb lemenyosával tud a felhasználó eljutni.

4. fejezet

Megvalósítás

A koncepció átbeszélése után áttérhetünk az implementációra. Egy Godot projekt Node-okból tevődik össze, amelyeket egy fa struktúrába rendezve tudunk kezelni. Érdekes egy Node-ra úgy tekinteni, mint egy osztályra, mivel valójában az is. Minden ilyen fát egy ún. "színteren" belül helyezünk el. Ez arra szolgál, hogy egyes részeit a projektünknek külön tudjuk kezelni a többitől, ez majd jobban világossá fog válni, amikor a játékos színteréről beszélek. Egy színtér felépítésében fontos szerepet játszik a Node hierarchia kialakítása. Vannak ugyanis Node-ok, amelyek megkövetelnek más Node-okat, mint gyermeket ahhoz, hogy rendesen működjön. Ilyen például egy StaticBody3D (egy statikus, fizikai erővel nem mozgatható test reprezentálására szolgál a háromdimenziós térben), amely megkövetel egy CollisionShape3D Node-ot (egy átlátszó polygonháló, amely ütközésvizsgálatra van használva). Érdekes megjegyezni, hogy ugyan gyakran a felhasználók előre elkészített Node-okat használnak a fejlesztés során, viszont a felhasználó maga is létrehozhat egy saját Node-ot, amely működését személyre szabhatja. Minden projekt egy "Main" színtér kiválasztásával kezdődik, ez az a színtér, amelyet a játékmotor mindig el fog indítani és elsőként meg fog jeleníteni.

4.0.1 Az alapok

Amint az előbb azt említettem a minden projekt egy main színtér kialakításával kezdődik, tehát én így is tettem. Az alap színtér a következő volt:

- Egy egyszerű sík, átmeneti textúrával,
- egy kezdetleges, textúra nélküli, kapszula alakú játékos modell,
- és egy pár akadály, amit a játékos mozgásának a tesztelésére használtam.

Majd ezt a színteret a szoftver fejlesztése alatt bővítettem különböző akadályokkal, amelyek már a küszöb-eseteket is tesztelni tudják.

A teszt színtér felállítása után létrehoztam egy külön színteret a játékos számára. Amint azt már említettem fontos elkülöníteni egymástól minden olyan objektumot, amely egy vagy több Node-ból áll össze külön színterekbe, ha azok nagyobb egységeket foglalnak magukba. Egyik hatalmas előnye ennek, hogy a projekt jobban átlátható lesz, és nem fognak keveredni a Node-ok egymással, valamint ezt a színteret külön tudom példányosítani, ha esetleg kódból szeretném meghívni a játékost valahol máshol.

Ezt a színteret a játékos gyökér Node-jának kiválasztásával kezdtem. A platformer játékoknál fontos az emberek két Node között szoktak választani: a `RigidBody3D`, és a `CharacterBody3D`, az én esetemben a választás a `CharacterBody3D`-re esett, azért, hogy miért a következő részben elmondom. Ennek egy kötelező része a már előbb említett `CollisionShape3D`, és a láthatóság szempontjából alárendeltem még egy `MeshInstance3D`-t is. A `CollisionShape3D`-t már elmondtam, hogy mi, de hogy egy helyen legyen a két magyarázat itt is megemlítem:

- **MeshInstance3D:** Egy háromdimenziós polygon hálót tartalmaz, kizárólag megjelenítési szempontja van, teljesen független az ütközésvizsgálattól.
- **CollisionShape3D:** Egy átlátszó polygon háló, amely az ütközésvizsgálathoz szükséges, független a `MeshInstance3D` által előállított polygon hálótól.

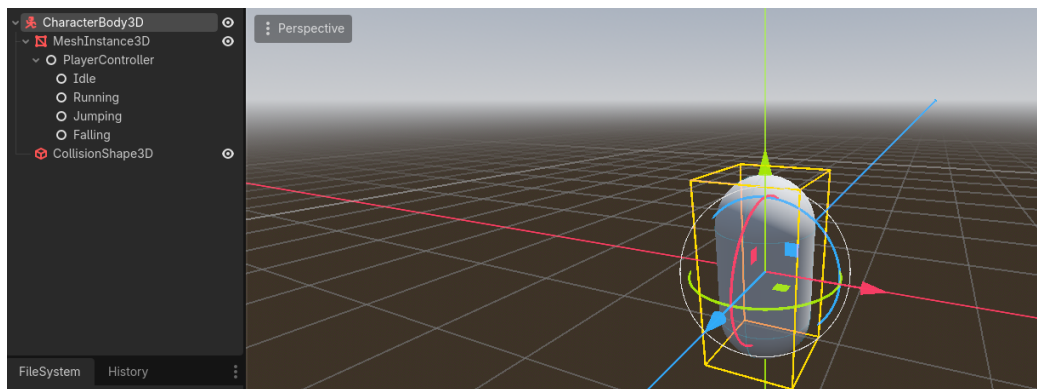
4.0.2 RigidBody vs CharacterBody

A játékos szülőjének két típust szoktak gyakran megválasztani platformerek esetén: a `CharacterBody3D`-t és a `RigidBody3D`-t. A lényeges dolog, amiben a kettő eltér egymásról, az hogy hogyan mozgathatod a testet. A `CharacterBody3D` típus esélyt ad neked arra, hogy egy saját fizikai motort implementálj, hogy a játékos a lehető legjobbnak éreződjön, míg a `RigidBody3D` a valós fizika szabályai szerint jár el. Egy játék esetében szerintem világosan látható, hogy nem a való világot szeretnénk szimulálni, hanem a lehető legjobb élményt szeretnénk nyújtani a játékosunknak, így olyanra írjuk meg a gravitációt, az ugrás magasságát, a gyorsulást, a súrlódást, stb, hogy az a játékos számára a legjobb játékérzést nyújtsa. A `RigidBody3D` típus alkalmazásával ezek a lehetőségek mind elvesznének és a Godot alap fizikai motorjára lennének utalva.

Valami, amit érdekesnek találtam miközben ezt a részt kutattam az az, amit Ludonauta írt a RigidBody vs characterbody: which one for your platformer?[11] című posztjában, ami így szól: "Your character isn't his body", ami először fúrcsán hangzik, de ez igaz. Mint ahogy azt már leírtam a CharacterBody3D független a polygon hálótól, amit alárendelsz, így mindegy, hogy ha kicseréled a modellt, minden más ugyan úgy működik. Az egyetlen dolog ami változtatna egy kicsit a működésén az az ütközésvizsgálathoz használt polygon háló alakjának a megváltoztatása lenne. Ezzel ugyan csak arra szeretnék visszautalni, mint amit Ludonauta is megjegyzett: használhatod mind a kettőt egy karakteren belül. Kell valami amihez valós fizikai hatások kellenek? rendeld a RigidBody alá a játékost! Nem szeretnél gravitációt? Rendeld a CharacterBody alá a játékost!

4.0.3 A játékos színtere

Az előzőekben elmondottakból tehát, a játékos színtere egyenlőre így néz ki: egy CharacterBody3D Node mint a gyökér a fába, egy MeshInstance3D és CollisionShape3D mint gyerek Node-ok, lásd: 4.1.



4.1. ábra: Képernyőkép a kezdetleges játékos színterről

Ahhoz, hogy a mozgásrendszert implementálni tudjam először el kellett gondolkoznom azon, hogy milyen más Node-ok fognak még esetleg kelleni nekem. Egyből tudtam, hogy kelleni fog egy kamera, ami a játékost követi, hogy láthassa a felhasználó, hogy mit is csinál. Továbbá, mint az a legtöbb játékban már megszokott és szinte elvárt, hogy a kamera kihasson a mozgás irányára. Így kibővítettem még három Node3D objektummal:

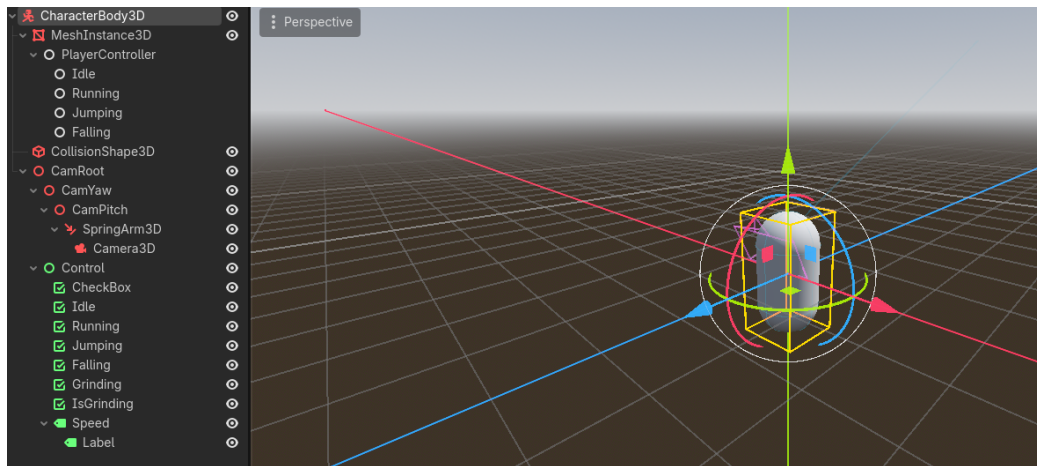
- **CamRoot:** A kamera csoport gyökere, nincs kihatással a többi részre, csak csoportosítás szempontjából van használva.
- **CamYaw:** A kamera oldalirányú elmozdulásának nyílvántartásáért felel.
- **CamPitch:** A kamera magassági elmozdulásának nyílvántartásáért felel.

A Node3D objektumok valójában pontok a háromdimenziós térben, így tökéletesen megfelelnek a különböző irányok nyílvántartására, ezért választottam inkább ezt a megoldást, mint azt, hogy változókat hozzak létre ezek figyelésére.

Létrehoztam továbbá még egy SpringArm3D és egy Camera3D Node-ot is.

- **SpringArm3D:** Ahogyan azt a nevéből is ki lehet venni, ez egy ún. "rugós kar", amely arra szolgál, hogy ha ütközik valamilyen akadállyal a színtérben, akkor mint egy rúgó elkezd összenyomódni. Ez arra kellett, hogy ha a kamera, ami ennek egy gyermeke, esetleg belemenne a falba, akkor ne okozzon bajt.
- **Camera3D:** Ez a Node maga a kamera. Kivetít akármit amit a legközelebbi nézőtérből (viewport) lát, a fát felfelé bejárva. Ha nincs nézőtér a fában, akkor a globális nézőteret fogja használni.

Annak érdekében, hogy az állapotok könnyen nyomonkövethetők legyenek csináltam egy pár CheckBox és Label Node-okból álló Control csoportot. A Control csoportok a Godot-on belül kétdimenziós síkok, amiket általában felhasználói felületnek szoktak használni. Ezt a csoportot a CamRoot alá rendeltem, így követni fogja a kamerát, ami annyit fog eredményezni, hogy a felhasználó képernyőjén folyamatosan láthatóak lesznek az elemek. Így egyenlőre a színtér az alábbi képen láthatóan néze ki[4.2].



4.2. ábra: A befejezett játékos színtere

4.0.4 Változtatások az állapotgéphez

Ahogy az már a [2. fejezet, 2.0.2. bekezdésében](#) is megadtam az automatánk két része a State és StateMachine osztályok nem változnak sokat.

A State osztályon belül:

- Hozzáadtam egy "TransitionEventHandler" signal-t (jelzőt), amellyel az állapotok közötti változást tudom szabályozni.
- A "StateMachine" osztályt átneveztem "PlayerController"-re.
- Hozzáadtam egy "gravity" változót, amelyet, akármelyik állapot el tud érni, ha szüksége lenne rá.
- Hozzáadtam egy "movementDirection" nevű változót, amely egy három-dimenziós vektor alakjában tárolja a felhasználói inputból kiszámított mozgási irányt.

Valamint az átnevezett PlayerController osztályon belül:

- Hozzáadtam egy "TransitionToEventHandler" nevű signal-t, ami a különböző állapotok közötti átmenetért felel.
- Hozzáadtam egy Control és egy új Dictionary osztályú objektumot, amik vizuális debug-olásért lettek létrehozva. Miután összegyűjtöttük az összes CheckBox Node-ot, amik kellenek a legelsőt már aktiváljuk is.

Az "OnTransitionTo" metóduson belül állítjuk a jelenleg megjelenített állapotot.

- Az "OnSetMovementState"" a jelenlegi mozgásállapotot állítja és az "OnSetMovementDirection" frissíti a jelenlegi állapot movementDirection változóját.

A PlayerController script-et hozzácsatoltam a megosztó nevű PlayerController Node-ra a játékos színterén belül. A State osztály meghagyjuk magát nem csatoltam hozzá semmihez, ezt csak arra használtam, hogy az állapotokat ebből az osztályból tudjam leszármaztatni.

4.0.5 Az állapotok megvalósítása

Ha visszaemlékeznek, a [3. fejezet, 3.0.6. szakaszában](#) vázoltam az állapotgépet és a különböző tervezett állapotokat és az azok közötti átmenetet. Most ezek megvalósításáról szeretnék beszélni. Mindegyik állapot implementációját röviden át fogom beszélni és a fontosabbakat ki fogom emelni és bele fogok menni a matekba is háttérben. A kódot nem minden esetben fogom ide beírni, viszont a CD mellékletben megtalálják a forráskódot, valamint a [CD Használati útmutató oldalon](#) elmondottak alapján tudják követni részletesebben a kódot.

A megvalósítás során az állapotokat egy időben írtam a játékos script-jével. Létrehoztam egy Player script-et és osztályt, amit a CharacterBody3D Node-hoz csatoltam. Ezen az osztályon belül hoztam létre azokat a metódusokat, amelyeket az állapotátmenetek vizsgálására használtam. Valamint itt számítottam ki a kamera mutatói irányát és a mozgásvektort is.

Az állapotok implementáció sorrendjében:

- **Idle:** Az Idle állapot azért felelős, hogy a mozgást megállítsa. Ha a játékos már mozgásban volt, akkor a beépített "MoveToward" metódus segítségével csökkenti a sebességét nullára. Ahhoz, hogy a játékos ide eljusson egy beépített metódust segítségével az "IsOnFloor"-ral megnézem, hogy a karakter a földön tartózkodik-e és, hogy e közben nincs-e letartva semelyik mozgás gomb.
- **Running:** A mozgás (vagy futás) állapotnál a játékos sebességét befolyásoljuk a movementDirection attribútummal, ami megad egy mozgás vektort. Ezt a vektort a "Player" osztályon belül és annál is fontosabban a _PhysicsProcess metóduson belül számítjuk ki. A _PhysicsProcess

egy felülírt metódus, ami az alap Node osztályból származik. Ezt a metódust hívja meg a Godot minden olyan frame előtt, amikor a motor fizikai számításokat végezne. Frame-nek, vagy keretnek hívjuk azt amikor a játékmotor újrarajzolja a képernyőt az új adatok szerint. Azért fontos ebben a metódusban kiszámítani a mozgásvektort, mivel nem szeretnénk minden frame-ben kiszámolni a játékos mozgásvektorát, ez csak számítási időt venne el, lassítaná a programot. Mivel felhasználjuk a CharacterBody3D "Velocity", avagy sebesség attribútumát, ezért a mozgás tulajdonképpen fizikai számításnak minősül. A mozgás számítása a következőképpen zajlik:

```
1  //...
2  Vector3 forward = _camera.GlobalBasis.Z;
3  Vector3 right = _camera.GlobalBasis.X;
4
5  var rawInput = Input.GetVector("Left", "Right", "
    Forward", "Backward");
6
7  _movementDirection = forward * rawInput.Y + right *
    rawInput.X;
8  _movementDirection.Y = 0.0f;
9  _movementDirection = _movementDirection.Normalized()
    ;
10 //...
```

Ha visszaemlékeznek említettem, hogy a kamera állását fel szeretném használni a mozgás kiszámításakor. Létrehozok két háromdimenziós vektort, a "forward"-ot és a "right"-ot, ezeket beállítom a kameránk globális bázisvektorának a Z és az X értékeire. A globális bázison belül minden objektumnak a helyi vektorjai ugyan abba az irányba mutatnak, így könnyen tudunk vele számolni. Az Input.GetVector-t ajánlja a Godot specifikáció az ún. "WASD" mozgáshoz, így azt használtam. Ebben az esetben visszkapunk egy kétdimenziós vektort, a pozitív és negatív X és Y tengelyek mentén, gondoljanak rá úgy, mint amikor egy kontrolleren egy joystick-et mozgatunk. Felmerülhet a kérdés, hogy minek szorozzuk meg a mozgásvektort még a kamera irányával is, nos, ha ezt a lépést nem tesszük meg akkor a karakterünk teljesen függetlenül fog mozogni a kamera állásától, ez azt fogja eredményezni, hogy az "Előre", ami a mi esetünkben a "W" gomb, mindig a globális bázis szerinti "Előre" lesz, tehát a Z irány, ami nagyon zavaros tud lenni a felhasználó számára. Viszont ha a vektorunk a kamerától függ akkor, ami a billentyűzetten előre lesz, az a monitoron is előre lesz. Miután megszoroztuk és összeadtuk a vektorjainkat fontos az Y ko-

ordinátákat nullára állítani másképpen ha felfele nézünk a kamerával akkor elrepülhetnénk. Az ugrást majd különsszedjük a saját állapotába. A mozgásvektorunkat egy normalizálással véglegesítjük, hogy megtartsuk az irányt, amibe mozgunk, de ne legyen a hossz miatt baj a későbbi számításokkor. Ugyanis ez még nem a vége, a Player osztályon belül továbbá megvizsgáljuk, hogy a jelenlegi helyzetben a játékos átmehet-e a "Running" állapotba, ennek a teljesítése csupán annyi, hogy a játékos a földön tartózkodjon és, hogy a "Movement" action-group-on belül az egyik gomb le legyen nyomva. Az action-group-ok, avagy cselekvés csoportok, billentyű vagy kontroller bemenetekre hallgató egységek csoportjai, ezt a programozónak kell megadnia.

```
1  // ...
2  if (Grounded)
3  {
4      if (Input.IsActionJustPressed("Movement"))
5      {
6          CurrentState = PlayerState.Running;
7      }
8      else if (_movementDirection.X == 0 &&
9               _movementDirection.Z == 0)
10     {
11         CurrentState = PlayerState.Idle;
12     }
13  }
14  // ...
```

Végül a "Running" állapoton belül beállítjuk a CharacterBody3D-nek a sebességét szintúgy a MoveToward metódussal, aminek a célvektorját beállítjuk a kiszámított mozgásvektorra szorozva a játékos sebességével (itt számít az, hogy a vektor normalizált-e vagy sem), és a változás gyorsaságát beállítjuk a játékos gyorsulására szorozva az előző frame kirajzolásával eltelt idővel, ezt hívjuk "deltának".

- **Jumping és Falling:** A "Jumping", avagy ugrás állapot relatíve egyszerű; amint a játékos lenyomja az "Ugrás" gombot, ami a mi esetünkben a "Space" vagyis szóköz, átlöködik az állapotgép a az ugrás állapotba, ahol a játékos sebességvektorának az Y koordinátájához hozzáadunk egy előre megszabott értéket, az ugrás erősségét. Ezek után az irányítást átadjuk a "Falling", avagy zuhanás állapotnak. Itt csökkentjük a játékos sebességvektorjának az Y koordinátáját a gravitáció szorozva a súllyal és a deltával. Valamint az esésnek átadjuk még a mozgásvektort, hogy a momentum, amivel a játékos felugrott megmaradjon és kapjon egy kis navigálási lehetőséget a levegőben.

- **Grinding:** Ez az állapot jelentette a legnagyobb kihívást számomra a projekt elkészítése során, de egyben ez volt a legélvezetesebb is. Kezdetnek beszéljük át az állapot funkcionalitását: a "grinding" a gördeszkázásból és görkorcsolyázásból eredt. A grind egy felület oldalán való végigcsúszás. Egyértelműen nem lenne jó, ha a játékos véletlen végig tudna csúszni minden objektum oldalán, így ezeket el kell különíteni, meg kell különböztetni, hogy ne legyen összezavaró a felhasználó számára. Ahhoz, hogy el tudjuk kezdeni a kód részét, elsőnek létre kell hozni egy ilyen felületet. Szerencsére a Godot-ban van egy előre beépített Node, amit erre fel tudunk használni, a Path3D és PathFollow3D. A Path3D Node tartalmaz egy Curve3D-t, amely valójában egy Bézier görbe, amit a PathFollow3D Node-ok tudnak használni arra, hogy bejárjanak. Egy Path3D Node-ot létrehozni egyszerű, csupán meg kell adni neki egy pár pontot, fontos, hogy az első pont egy null-vektor legyen, másképpen a számításokba hiba léphet, mivel a PathFollow3D által mozgatott Node-ok relatívak a PathFollow3D Node-hoz képest. Magának a Path3D-nek vagy PathFollow3D-nek nincs alaphálózati ütközési hálózata, így gyöker Node-ként egy StaticBody3D-t választottam, amihez hozzá tudok rendelni egy ütközési hálót. Szerencsére nem kell manuálisan létrehozni sem egy modellt, sem magát az ütközési hálót, mert a Godot-nak a CSGPolygon3D Node-ja támogatja, hogy egy Path3D Node-ot alapul véve generáljon annak mentén egy modellt, amelynek utána mind a polygon, mind az ütközési hálóját be tudjuk égetni a színtérbe. Azért fontos, hogy ütközést tudjunk vizsgálni, mert az implementációm arra hagyatkozik, hogy a játékos karakter lábához van csatolva egy téglatest alakú ütközési háló, ami folyamatosan figyeli, hogy esetleg nem egy csúszható felülettel érintkeztünk-e. Ezt a Player osztály "IsGrinding" metódusán keresztül tesszük, először meggyőződünk arról, hogy maga a háló ütközik-e egyáltalán valamivel, valamint, hogy az objektum, amivel ütköztünk a RailGrinding osztályhoz tartozik-e.

```
1  // ...
2  public bool IsGrinding()
3  {
4      return _shapecast.IsColliding() && _shapecast.
          GetCollider(0) is RailGrinding;
5  }
6  // ...
```

A RailGrinding a világban elhelyezett csúszható felületeknek az osztálya ennek nem csak megkülönböztető szerepe van, de ezen osztály metódusával

számítjuk ki, hogy a játékos mely beégetett pontra landolt és, hogy honnan tudja elkezdni a csúszást. Ezt a `CalculateTargetRailPoint` metódusban a következőképpen tesszük:

```
1  public void CalculateTargetRailPoint(Vector3
    playerPosition)
2  {
3      float progress = path.Curve.GetClosestOffset(
    playerPosition - path.GlobalPosition);
4
5      Transform3D sample = path.GlobalTransform * path.
    Curve.SampleBakedWithRotation(progress);
6
7      pathFollow.Progress = progress;
8      pathFollow.GlobalTransform = sample;
9
10     pathFollow.ResetPhysicsInterpolation();
11 }
```

Először is a `PathFollow3D`-hez tartozó attribútum az ún. "Progress", ami háromdimenziós egységekben adja meg a már megtett utat egy `Path3D` objektumon, ezt a `Path`-hez tartozó `Curve3D` metódusával a "GetClosestOffset"-tel kiszámítjuk. Ennek a metódusnak meg kell adni, hogy melyik ponthoz szeretnénk a legközelebbi eltolást megkapni (ez a "toPoint"), ezt pedig a játékos test globális pozíciójának és a `Path` globális pozíciójának a különbségeként fogjuk definiálni, mivel a `Path` globális pozíciója az összes beégetett pont legelső pontja lesz, így könnyen meg tudjuk mondani, hogy a játékos milyen messze van a kezdeti ponttól. Ezután a létrehozunk egy "sample" változót, ahol a kiszámítom a `PathFollow3D`-nek, hogy pontosan milyen utat kövessen. A `Transform3D` osztály csupán egy osztály alá vonja a globális pozíciót, a globális forgást és az objektum skáláját. A pontosabb számítás érdekében miután lekértem a beégetett pontokat ezt még megszorozom magának a `Path`-nek a globális transzformációjával. Ezek után az egyes adatokat rögzítem és alaphelyzetbe állítom a fizikai interpolációkat. Ez ugyan nem szükséges, viszont sok helyen ajánlották, hogy ha esetleg túl éles szögből közelítené meg a játékos a pontot, akkor ne okozzon esetleges vizuális hibákat. Amint a `Player`-en belül az "IsGrinding" igazat ad vissza, az állapotgép átkerül a `Grinding` állapotba. Elsőnek itt is megvizsgáljuk, hogy érintkezik-e olyan felülettel, ami nekünk kell, mivel a váltások között esély van arra, hogy az ilyen felületet elhagyjuk és beragadunk. Miután ezen átmentünk meghívjuk a jelenleg felhasznált felületnek a `CalculateTargetRailPoint` metódusát, amiben a `PathFollow` értékeit kiszámoljuk. Ezután fizikai számításokként kiszámoljuk az

eddig megtett utat a következőképpen:

```
1  // ...
2  float progress = _usedRail.pathFollow.Progress + -
    _usedRail.pathFollow.Basis.Z.Normalized().Dot(
        parent.Velocity.Normalized()) * parent.Velocity.
        Length() * grindSpeed * (float)delta;
3  _usedRail.pathFollow.Progress = progress;
4  // ...
```

A már meglévő úthoz hozzáadjuk a PathFollow bázisvektorának Z normalizáltját (mivel nem érdekel minket az, hogy milyen hosszú a vektor, csak az, hogy melyik irányba mutat), aminek utána vesszük a mínusz egyszeresét, mivel a Godot-n és a legtöbb játékmotoron belül az "előre" mutató vektor a -Z. Ezt utána skalárisan összeszorozzuk a játékos normalizált sebességvektorával. Ez a skaláris szorzat fogja megmondani nekünk a csúszás *irányát*. Ezek után megszorozzuk még a játékos sebességvektorának a hosszával, a "grindSpeed" változóval, valamint a delta számmal. Mindezt azért, hogy legyen valami sebessége is a csúszásnak.

```
1  // ...
2  parent.GlobalPosition = _usedRail.pathFollow.
    GlobalPosition;
3  parent.GlobalRotation = _usedRail.pathFollow.
    GlobalRotation;
4  // ...
```

A számítások után a játékost transzformáljuk a PathFollow helyzete és forgása alapján. Majd a módosítjuk a sebességét:

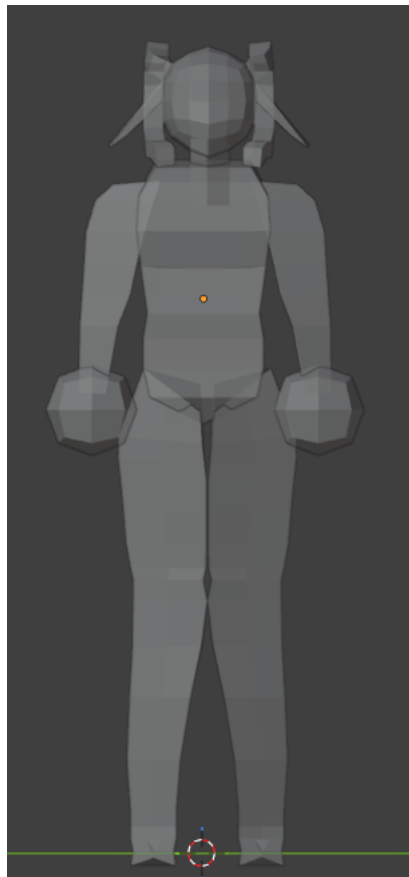
```
1  // ...
2  parent.Velocity = -_usedRail.pathFollow.Basis.Z.
    Normalized() * parent.Velocity.Length() * (-
        _usedRail.pathFollow.Basis.Z.Normalized()).Dot(
        parent.Velocity.Normalized());
3  parent.Velocity += -_usedRail.pathFollow.Basis.Z.
    Normalized() * (-_usedRail.pathFollow.Basis.Z.
        Normalized()).Dot(-Vector3.Up.Normalized()) * 5f
        * (float)delta;
4  // ...
```

Elsőnek beállítjuk a jelenlegi irány és sebesség szorzatával a játékos sebességét, majd hozzáadjuk a gravitációs ellenállást, hogy ha túl meredek lenne a csúszás iránya, akkor visszafele induljon meg. A gravitációs ellenállás a $(-Vector3.Up.Normalized()) * 5f$ lenne. A Vector3

osztálynak a felfele mutató értéke az Y , ennek vesszük a mínusz egyszerűsét, tehát már lefele mutat, normalizáljuk, így a hosszat mi állíthatjuk be, és ezt is tesszük azzal az ötös szorzóval. A skaláris szorzat itt szintén az irány miatt kell, hogy minden esetben a gravitációs vektor lefele mutasson.

4.0.6 Modellezés és Textúrázás

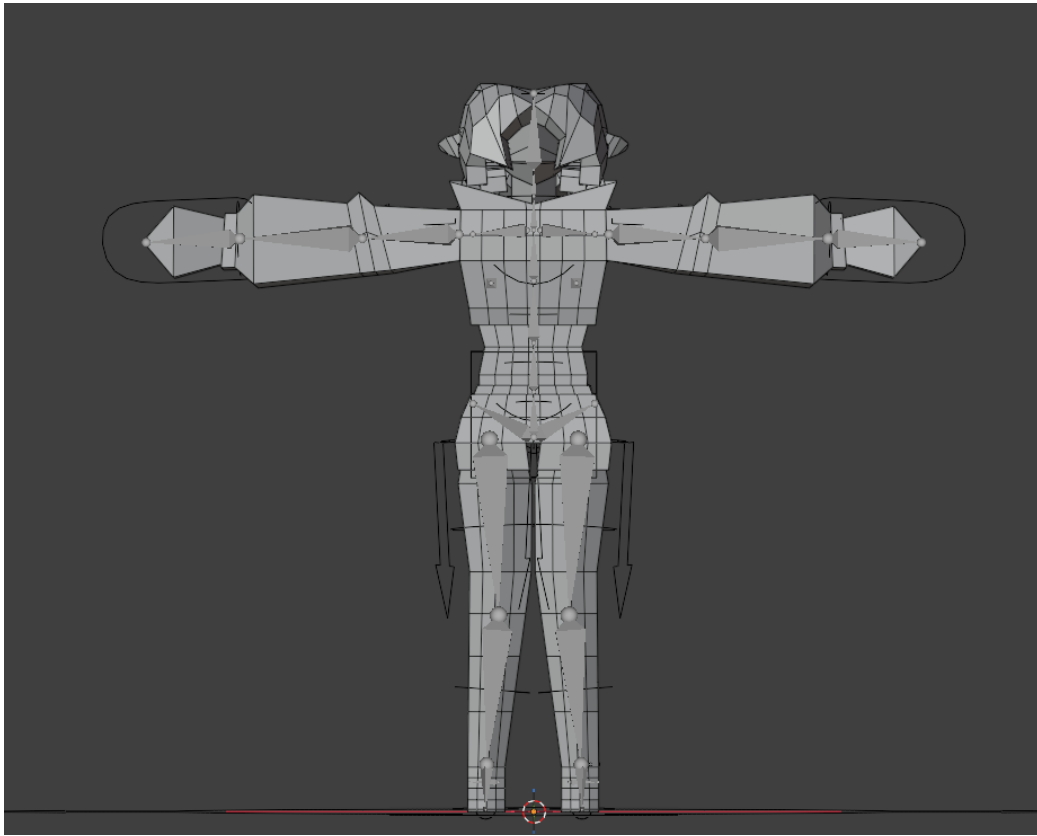
Amint azt már a [3 fejezet 3.0.2 alfejezetében](#) is említettem a modellezéshez a Blender nevű ingyenes, nyílt forráskódú 3D modellező programot használtam. Itt a karakter két iteráción esett keresztül. Az első iteráció, ahogy az alábbi képen is látható: [\[4.3\]](#), nem jutott el sokáig és nem a legigényesebb.



4.3. ábra: Az első iteráció

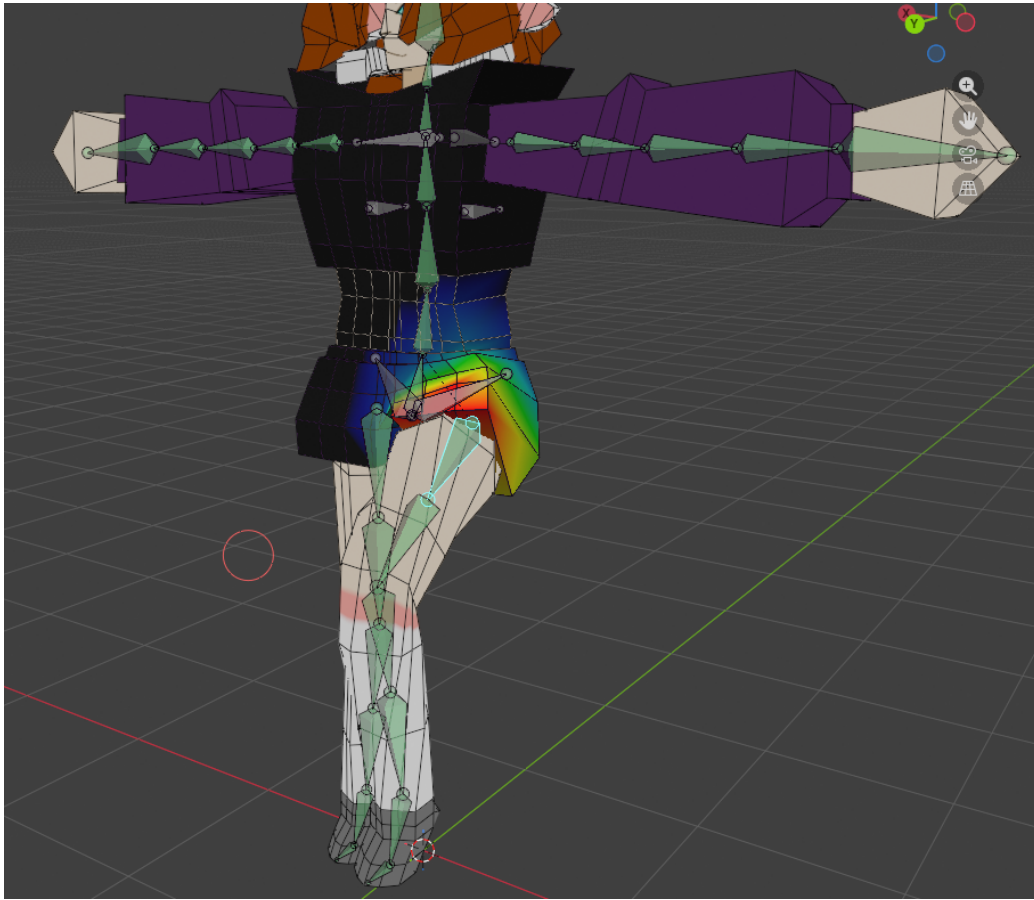
Itt még újra kellett tanulnom a Blender alapjait és a Low-poly modellezés szabályait. Ebben nagyon segített Crashune Academy: Full Blender

Character Modeling Tutorial[8] sorozata. Viszont a tesztelés során sokáig ez a modell volt használva. Amikor elkezdtem a vége felé közeledni a projektnek illőnek éreztem egy jobb modellt készíteni és a helyett, hogy ezt a modellt folytattam volna inkább kezdtem egy újat. Néha jobb csak újból kezdeni. Viszont örömmel jelentem be, hogy megérte, mivel a második iteráció sokkal jobban sikerült mint az első! A modell topológiájára sokkal jobban odafigyeltem és eltakarítottam a nem használt pontokat, szerintem az alábbi kép ezt jól átadja: [4.4].



4.4. ábra: A második iteráció

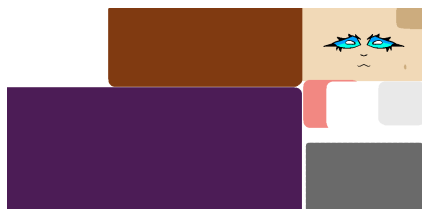
Ezen a verzión már ún. "rig" is van, ami a háromdimenziós animációnál annyit tesz, hogy a modell rendelkezik csontokkal, amely csontokhoz környékén lévő pontokhoz hozzá vannak rendelve értékek, ún. "súlyok". Ezek a súlyok mondják meg, hogy az egyes csontok mennyire befolyásolhatják az azon a ponton történő deformációt. A Blender legújabb változatában elérhető egy "Rigify" nevű kiegészítő, ami ezt a folyamatot egyszerűbbé teszi. Generál egy csont struktúrát, amelyet hozzá kell illeszteni a modellhez és a többit a



4.5. ábra: Példa a csontok súlyozására

kiegészítő elvégzi. Meg kell jegyezni, hogy ez nem a legjobb minőséget vagy eredményt adja, de gyors és könnyen szerkeszthető. A Blender-ön belül a súlyok színekként jelennek meg, pirostól feketéig. A piros a legnagyobb befolyást, míg a fekete a legkisebb (nem létező) befolyást jelenti és az e között lévő színek eloszlanak, ahogyan az alábbi képen is látható: [4.5].

Végül a modell textúráit úgy döntöttem, hogy egy textúra atlaszként fogom tárolni, lásd: [4.6], mivel eléggé egyszerűre csináltam a szín palettáját. A textúra atlaszok olyan képek, amelyek egy modell teljes textúráját tartalmazzák. Tehát nem külön-külön képekre vannak szétbontva. Így a végső modellt a következő képen láthatják: [4.7].

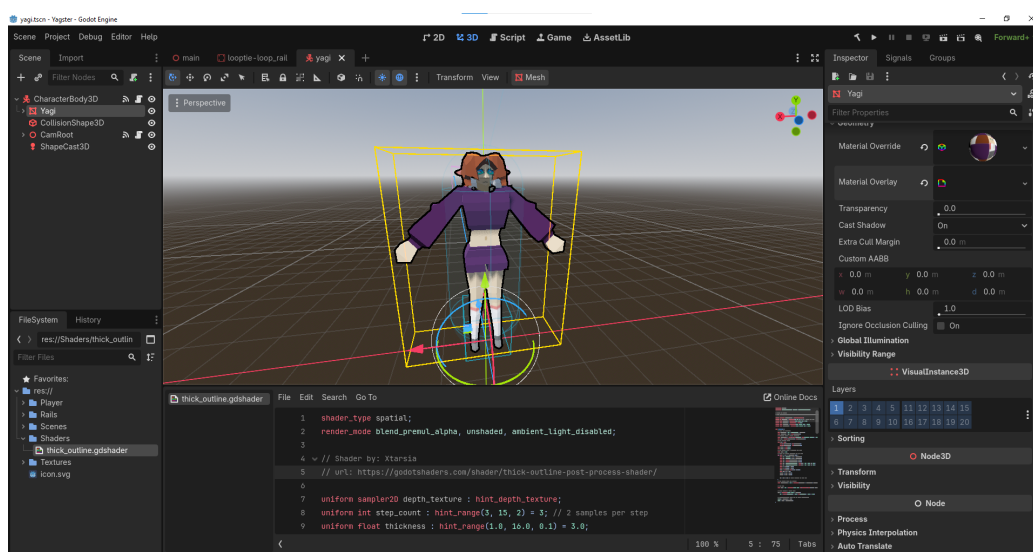


4.6. ábra: A modell textúra atlasza



4.7. ábra: A textúrázott modell

A játékos színterén belül kicseréltem a régi modellt, behelyeztem az újat és hozzáadtam a geometriához egy shader-t, hogy jobban illjen ahhoz a JSR / képregény témához, amire hajaztam[4.8]. A shader-t nem én írtam, ennek a készítője Xtarsia a GodotShaders-ön publikálta[14].



4.8. ábra: A végső modell shader-el együtt a Godot szerkesztőjében

Irodalomjegyzék

- [1] Audacity's website. <https://www.audacityteam.org/download/>. Accessed: 2026.03.01.
- [2] Blender's website. <https://www.blender.org/download/>. Accessed: 2026.03.01.
- [3] Draw.io's website. <https://www.drawio.com/>. Accessed: 2026.03.01.
- [4] Godot's website. <https://godotengine.org/>. Accessed: 2026.05.06.
- [5] Krita's website. <https://krita.org/en/>. Accessed: 2026.03.01.
- [6] Lmms's website. <https://lmms.io/>. Accessed: 2026.03.01.
- [7] Slking2pixel's itch.io page. <https://captain4lk.itch.io/slk-img2pixel/>. Accessed: 2026.03.01.
- [8] Crashune Academy. Full blender character modeling series. https://www.youtube.com/playlist?list=PLcaQc6eQjXCxWmn5jxE_GT0ft9ZxChu1. Published: 2025.
- [9] Charles Burgar. How omnimovement works in black ops 6. 2024.
- [10] Georges Ltief. Finite state machines: An introduction to fsms and their role in computer science. 2024.
- [11] Ludonauta. Rigidbody vs characterbody: Which one for your platformer? 2025.
- [12] Tom Mancini. Jet set radio - tokyo in cel-shade. 2023.
- [13] Team Reptile. Bomb rush cyberfunk. 2020.
- [14] Xtarsia. Thick outline post process shader. <https://godotshaders.com/shader/thick-outline-post-process-shader/>. Accessed: 2026.05.06.